Matrix-Residual Graph Neural Networks for Graph Classification

Xuelin Hu[1] and Xiaoqin Fu[2]

[1]School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China
[2]School of Railway Locomotive and Rolling Stock, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China

Corresponding author:
Xiaoqin Fu[2]

Email address: fuxiaoqin@ltzy.edu.cn

ABSTRACT

Background. Residual connections are widely used to stabilize graph neural networks, but residual design is often treated as a one-dimensional depth-wise choice. Multi-branch graph classifiers create a richer design space: residual information can flow along depth, across branches, or through a local two-dimensional branch-by-layer neighborhood.
Methods. We study Matrix-Residual Graph Neural Networks as a controlled residual-topology framework for graph classification. The model family represents hidden states on a branch-by-layer grid and compares five information-flow designs: plain message passing, vertical residual reuse, horizontal branch-wise exchange, local matrix residual reuse, and sparse/gated matrix reuse. The main benchmark evaluates six datasets, four message-passing operators, five model families, and five graph-level folds under a shared training protocol. We further run branch-count ablations, fold-0 sensitivity scans, five-fold checks of selected MatrixResGated candidates, and mechanism analyses based on branch diversity, branch cosine similarity, branch CKA, gradient norms, residual activity, and learned gates.
Results. No single residual topology dominates all datasets and operators. Across the 24 dataset–operator combinations, MatrixRes obtains the largest number of wins (9/24) and the best mean rank, followed by MatrixResGated. Under GCNConv, the winners differ by dataset: Plain is strongest on PROTEINS, HorizontalRes is strongest on DD, MatrixResGated is strongest on the low-accuracy ENZYMES setting, and MatrixRes is strongest on MUTAG, AIDS, and Mutagenicity. Branch-count ablations on PROTEINS and DD reveal dataset-dependent operating regions rather than monotonic capacity gains. Mechanism metrics are consistent with the view that useful branch expansion needs representational separation without excessive residual traffic or weakened gradients. Five-fold tuned-candidate checks show that a smaller MatrixResGated hidden dimension reduces parameters on DD while retaining competitive accuracy; the PROTEINS sparsity candidate remains variable across folds.
Conclusions. Residual topology, branch count, and residual filtering should be selected jointly. Additional branches help only when they create useful functional diversity; otherwise, residual traffic and branch redundancy can offset the added capacity. Matrix-style reuse is best understood as a tunable residual family rather than a universal replacement for simpler vertical or horizontal residual paths.

1 INTRODUCTION

Graph neural networks (GNNs) learn graph representations by repeatedly aggregating information from local neighborhoods (Gori et al., 2005; Kipf and Welling, 2017). This message-passing view has become a standard tool for graph classification because it combines node attributes, local connectivity, and graph-level structure in a single differentiable model (Gilmer et al., 2017; Wu et al., 2021). In molecular and protein-oriented benchmarks, graph labels may depend on both local motifs and global organization, so useful intermediate evidence must survive until graph-level pooling (Duvenaud et al., 2015; Ying et al., 2018).

Increasing depth or adding parallel branches can expand model capacity, but it also changes how information is transported through the network. Deep GNNs may suffer from over-smoothing, where node

representations become too similar across propagation steps (Li et al., 2018; Oono and Suzuki, 2020). They may also suffer from over-squashing, where long-range information is compressed into limited-size messages (Alon and Yahav, 2021; Topping et al., 2021). These issues are especially relevant for graph-level prediction because the final readout depends on the quality of the intermediate representations that reach the pooling stage.

Residual connections are a common response to these difficulties. They improve gradient flow and allow earlier representations to be reused, a principle established in deep convolutional networks and later adapted to graph architectures (He et al., 2016; Bresson and Laurent, 2017). Other depth-control strategies, including DropEdge, Jumping Knowledge aggregation, and graph-specific normalization, also aim to preserve useful information across propagation steps (Rong et al., 2019; Xu et al., 2018). However, residual connectivity is still commonly treated as a default layer-wise component rather than as the object of study. Most residual graph classifiers reuse information along depth in the same stream.

Once a model contains multiple branches, depth-wise residual reuse is only one possible route. Residual information may also move across neighboring branches at the same depth or through diagonal branch-depth neighborhoods. Existing graph-classification work has explored stronger message-passing operators, hierarchical pooling, attention, and higher-order neighborhood mixing (Abu-El-Haija et al., 2019; Bianchi et al., 2020). These designs improve the expressive or aggregation capacity of GNNs, but they do not isolate the residual-routing question: if several branch states are available at a given depth, should the model reuse only the previous state of the same branch, exchange states laterally, or combine both directions in a local two-dimensional residual neighborhood?

This paper studies residual reuse as a structured design problem on a branch-by-layer grid. The central question is not whether skip connections should be present, but where residual signals should flow. A grid view exposes three controlled neighborhoods: vertical reuse along depth, horizontal exchange across neighboring branches, and matrix-style reuse that combines same-branch, neighboring-branch, and diagonal residual paths. A sparse/gated matrix variant further tests whether matrix residual traffic should be filtered before injection. This framing follows controlled benchmark practice: the goal is not to add many unrelated architectural components, but to make one design axis explicit enough to evaluate (Keriven et al., 2020; Morris et al., 2020).

The study is deliberately controlled. It fixes the graph-classification pipeline within each comparison, evaluates the same architecture family under several canonical message-passing operators, and changes only the residual topology and residual-control mechanism. This separation keeps the empirical comparison focused on information-flow design rather than on unrelated changes in readout structure, task-specific feature engineering, or operator-specific tuning. Recent graph models may achieve stronger absolute performance by changing several components simultaneously; here, those components are intentionally held fixed so that the residual topology itself remains identifiable.

The work is organized around three research questions:

1. **Which residual topologies are preferred under a shared graph-classification protocol?** We compare plain message passing, vertical residual reuse, horizontal branch-wise exchange, local matrix residual reuse, and sparse/gated matrix reuse while holding the remaining pipeline fixed.

2. **How does branch count change residual behavior?** We test whether additional branches improve accuracy monotonically or only within a useful operating region, and we relate these trends to diversity, similarity, gradients, and residual traffic.

3. **When does controlled matrix reuse help?** We test whether matrix residual traffic should be passed densely or filtered, and whether candidate residual-control settings remain valid after full-fold reruns.

The main contributions are:

- **A branch-by-layer formulation of residual graph classifiers.** Vertical, horizontal, matrix, and sparse/gated matrix residual families are expressed as local neighborhoods on the same grid, making their information-flow differences explicit.

- **A controlled evaluation of residual topology rather than a new message-passing operator.** The benchmark keeps the classifier, readout, fold protocol, and backbone choices matched so that performance differences can be attributed to residual routing.

- **A mechanism-based interpretation of branch expansion.** Accuracy is analyzed together with branch diversity, branch cosine similarity, branch CKA, gradient norms, and residual activity, linking performance changes to branch separation, residual traffic, and optimization behavior.

- **A conservative validation protocol for residual-control settings.** Sensitivity scans are used only to identify candidates, and selected settings are rerun across folds before being treated as empirical evidence.

2 MATERIALS AND METHODS

This section defines the graph-classification task, the Matrix-Residual architecture family, the benchmark protocol, and the mechanism metrics. The guiding principle is to isolate residual topology while holding the remaining learning pipeline fixed.

2.1 Problem definition and notation

Let $\mathscr{G} = (\mathscr{V}, \mathscr{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$. For a dataset $\mathscr{D} = \{(\mathscr{G}_i, y_i)\}_{i=1}^{N}$, the objective is to learn $f_\theta : \mathscr{G} \to \hat{y}$, where $\hat{y}$ is the predicted graph label.

The compared model families share the same graph-classification wrapper:

- **Plain**: stacked message passing without explicit residual reuse.

- **VerticalRes**: same-branch residual reuse from the previous layer.

- **HorizontalRes**: cross-branch exchange between adjacent branches at the same depth.

- **MatrixRes**: local two-dimensional reuse over vertical, horizontal, and diagonal neighbors.

- **MatrixResGated**: matrix reuse with gated or sparse residual filtering.

2.2 Shared graph-classification pipeline

Let B be the number of branches and L be the number of message-passing layers. The hidden state at branch b and layer ℓ is written as $\mathbf{H}_b^{(\ell)}$, where $b \in \{1, \ldots, B\}$. A branch-local propagation step is

$$\mathbf{Z}_b^{(\ell)} = \phi_b^{(\ell)} \left(\mathbf{H}_b^{(\ell-1)}, \mathbf{A} \right), \tag{1}$$

where ϕ is instantiated as one of four standard operators: GCNConv (Kipf and Welling, 2017), GATConv (Veličković et al., 2018), SAGEConv (Hamilton et al., 2017), or GINConv (Xu et al., 2019). The residual-enhanced update is

$$\mathbf{H}_b^{(\ell)} = \mathbf{Z}_b^{(\ell)} + \sum_{(b',\ell') \in \mathscr{N}(b,\ell)} \mathscr{R}_{(b',\ell') \to (b,\ell)} \left(\mathbf{H}_{b'}^{(\ell')} \right). \tag{2}$$

The only term that changes across architectures is the residual neighborhood $\mathscr{N}(b,\ell)$ and the residual transform $\mathscr{R}$.

After the final message-passing layer, each branch is pooled to a graph-level vector by global mean pooling and the branch vectors are combined before classification. We intentionally avoid architecture-specific pooling heads such as differentiable pooling or self-attention pooling (Ying et al., 2018; Lee et al., 2019), because the purpose is to isolate residual topology rather than to compare readout mechanisms. The classifier is trained with cross-entropy loss. Figure 1 summarizes the shared pipeline and the residual routes compared in the study.

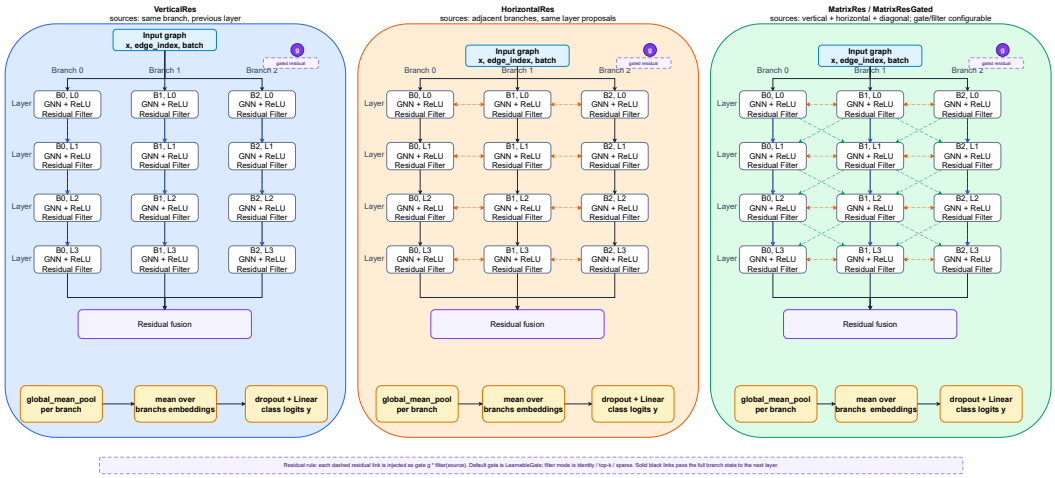

Figure 1. Architecture overview of the compared residual neighborhoods on the branch-by-layer grid. The same message-passing backbone and graph-level readout are kept fixed while the residual routes are changed.

2.3 Residual neighborhoods

Plain. The plain baseline has no residual neighborhood:

$$\mathcal{N}(b,\ell) = \emptyset. \tag{3}$$

VerticalRes. Vertical residual reuse injects the previous layer on the same branch:

$$\mathcal{N}(b,\ell) = \{(b,\ell-1)\}. \tag{4}$$

HorizontalRes. Horizontal residual reuse exchanges information across adjacent branches at the same depth:

$$\mathcal{N}(b,\ell) = \{(b-1,\ell),(b+1,\ell)\}, \tag{5}$$

with boundary branches using only valid neighbors. When $B = 1$, this horizontal neighborhood is empty.

MatrixRes. Matrix residual reuse combines vertical, horizontal, and diagonal neighborhoods:

$$\mathcal{N}(b,\ell) = \{(b,\ell-1),(b-1,\ell),(b+1,\ell),(b-1,\ell-1),(b+1,\ell-1)\}. \tag{6}$$

This local stencil is intentionally not dense all-to-all mixing. It preserves an interpretable residual structure over the branch-by-layer grid. Invalid boundary neighbors are removed; therefore, when $B = 1$, MatrixRes keeps the same-branch vertical residual path after the first layer but has no branch-neighbor routes.

MatrixResGated. MatrixResGated uses the same neighborhood as MatrixRes, but the residual transform controls the injected signal:

$$\mathscr{R}(\mathbf{U}) = \begin{cases} \alpha\mathbf{U}, & \text{identity residual filtering with a gate,} \\ \alpha\,\mathrm{softshrink}_{\lambda}(\mathbf{U}), & \text{sparse residual filtering with a gate.} \end{cases} \tag{7}$$

In the main benchmark, Plain, VerticalRes, HorizontalRes, and MatrixRes use identity residual filtering. MatrixResGated uses sparse residual filtering with $\lambda = 0.05$ and a learnable scalar gate initialized at 0.8. When $B = 1$, MatrixResGated still applies gated sparse filtering to the remaining matrix-residual paths. The implementation also supports top-k residual filtering and fixed gates, but these are not the default settings used for the main reported results.

2.4 Datasets and evaluation protocol

The main benchmark uses six datasets from TUDataset (Morris et al., 2020): PROTEINS, DD, ENZYMES, MUTAG, AIDS, and Mutagenicity. PROTEINS and DD are binary protein-structure graph classification benchmarks, ENZYMES is a six-class enzyme classification benchmark, and MUTAG, AIDS, and

Mutagenicity provide supplementary molecular-graph robustness checks. All datasets are loaded through PyTorch Geometric (Fey and Lenssen, 2019). All experiments use the raw benchmark features and graph structures without task-specific feature engineering, graph augmentation, or edge redesign.

All main results use graph-level five-fold evaluation. For fold k, the test accuracy is

$$\mathrm{Acc}_k = \frac{1}{|\mathscr{T}_k|} \sum_{(\mathscr{G}_i, y_i) \in \mathscr{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \tag{8}$$

Across folds, we report

$$\overline{\mathrm{Acc}} = \frac{1}{5} \sum_{k=1}^{5} \mathrm{Acc}_k, \tag{9}$$

and the fold-level standard deviation.

2.5 Training configuration

All main benchmark runs use the default branch count $B = 3$. The benchmark is repeated under four message-passing operators: GCNConv, GATConv, SAGEConv, and GINConv. The hidden dimension is 64 unless a sensitivity setting changes it. The default depth is $L = 4$ message-passing layers, except DD, where $L = 3$ is used for the main benchmark. All models use ReLU nonlinearities and the default PyTorch/PyTorch Geometric parameter initialization for the corresponding layers. Models are optimized with Adam (Kingma and Ba, 2014); the checkpoint with the lowest validation loss is retained for test evaluation. Early stopping monitors validation loss, and all runs use ReduceLROnPlateau with factor 0.5, patience 15, and minimum learning rate 10^{-5}. Gradients are clipped at norm 2.0. The branch-count ablation evaluates $B = 1, \ldots, 8$ for HorizontalRes, MatrixRes, and MatrixResGated on PROTEINS and DD under GCNConv. The sensitivity scan evaluates MatrixRes and MatrixResGated on fold 0 at $B = 3$, varying learning rate, dropout, hidden dimension, sparsity strength, and gate initialization. Selected MatrixResGated candidates are then rerun across all five folds.

Table 1. Key experimental settings shared by the main benchmark.

Parameter	Value
Message-passing operators	GCNConv, GATConv, SAGEConv, GINConv
Default branch count	3
Default layers	4, except DD uses 3
Default hidden dimension	64
Evaluation	Five-fold graph classification
Branch-count ablation	$B = 1, \ldots, 8$
Primary metric	Mean best test accuracy
Sensitivity scope	Fold 0, $B = 3$
Plain/VerticalRes/HorizontalRes	200 epochs, patience 60, learning rate 0.005, weight decay 10^{-4}, dropout 0.5
MatrixRes/MatrixResGated	240 epochs, patience 80, learning rate 0.003, weight decay 5×10^{-5}, dropout 0.3
Dataset-specific settings	DD uses dropout 0.3; ENZYMES uses batch size 128
Learning-rate schedule	ReduceLROnPlateau, factor 0.5, patience 15, min lr 10^{-5}
Gradient clipping	Norm 2.0

2.6 Mechanism metrics

To interpret branch-count behavior, we track several mechanism indicators on the final branch-level graph embeddings. Branch diversity is the mean pairwise Euclidean distance between branch embeddings. Branch cosine similarity is the mean pairwise cosine similarity, and branch CKA is linear centered-kernel alignment computed on the same branch-embedding set. For $B = 1$, there is no branch pair; we report diversity as 0 and cosine and CKA as 1 under the single-branch self-similarity convention. Mean gradient norm is the representative parameter-gradient norm recorded during training. Residual-count totals summarize how many residual inputs are available across residual sites, active ratio measures the fraction

of nonzero residual entries after filtering, and the gate statistic reports the learned scalar gate used by MatrixResGated. These metrics are used as correlational diagnostics rather than as causal tests.

3 RESULTS

All results are reported as five-fold mean best test accuracy ± fold-level standard deviation unless otherwise stated. The source tables are the summaries under `records/LATEST`; the main benchmark contains 600 completed runs.

3.1 Overall benchmark performance

The main benchmark asks whether residual topology has a stable winner across datasets and message-passing operators. It does not. Table 2 summarizes the full benchmark: across the 24 dataset–operator combinations, MatrixRes is the most frequent winner, with 9 wins and the best mean rank (2.12). MatrixResGated and Plain each win 5 combinations, HorizontalRes wins 3, and VerticalRes wins 2. Thus, matrix-style reuse has the most favorable aggregate profile in this controlled benchmark, but every topology has a setting in which it is the strongest choice.

Table 2. Winner counts and mean ranks across the 24 dataset–operator combinations. Lower mean rank is better.

Model	Wins	Mean rank	Interpretation
Plain	5	3.04	Strong on selected small or easier settings
VerticalRes	2	3.38	Depth-only reuse helps selectively
HorizontalRes	3	3.54	Competitive when lateral exchange is sufficient
MatrixRes	**9**	**2.12**	Most frequent winner in this benchmark
MatrixResGated	5	2.92	Useful when sparse control helps residual traffic

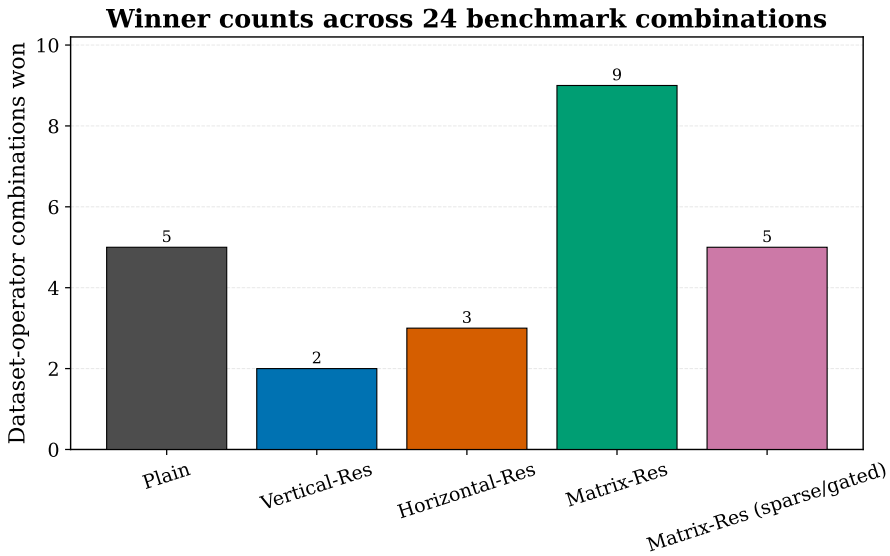

Figure 2. Model-level winner counts across the 24 dataset–operator combinations.

Table 3 gives the cell-level evidence behind the win counts in Table 2. The matrix family is most prominent on MUTAG, AIDS, and Mutagenicity, where MatrixRes or MatrixResGated wins most operator settings. The protein-oriented datasets are more mixed: PROTEINS alternates between Plain and HorizontalRes, while DD alternates among HorizontalRes, MatrixRes, and VerticalRes. ENZYMES remains difficult and does not produce a stable residual-topology preference.

The ENZYMES outcomes should be interpreted cautiously because the absolute accuracies are low for all compared models. Keeping ENZYMES in the full 24-combination summary preserves the

Table 3. Winning model for each dataset and message-passing operator. Values in parentheses are mean best test accuracy.

Dataset	GCNConv	GATConv	SAGEConv	GINConv
PROTEINS	Plain (0.7044)	HorizontalRes (0.7116)	Plain (0.6999)	HorizontalRes (0.7080)
DD	HorizontalRes (0.7181)	MatrixRes (0.7164)	MatrixRes (0.7198)	VerticalRes (0.7224)
ENZYMES	MatrixResGated (0.2933)	Plain (0.2717)	VerticalRes (0.2883)	Plain (0.3100)
MUTAG	MatrixRes (0.7609)	MatrixResGated (0.7555)	MatrixRes (0.7504)	MatrixResGated (0.8081)
AIDS	MatrixRes (0.8365)	MatrixRes (0.8875)	Plain (0.8850)	MatrixResGated (0.9280)
Mutagenicity	MatrixRes (0.7784)	MatrixRes (0.7909)	MatrixResGated (0.8031)	MatrixRes (0.8024)

preregistered benchmark scope, but it should not be read as strong evidence for a residual topology by itself. Excluding ENZYMES leaves MatrixRes with 9 wins across 20 dataset–operator combinations and a mean rank of 2.05, so the aggregate MatrixRes preference remains. The split is still context-dependent: MatrixRes is most prominent on the molecular datasets, whereas the protein-oriented datasets produce mixed winners.

3.2 GCNConv benchmark slice

Table 4 reports the full GCNConv slice at the default branch count $B = 3$. This controlled slice is useful because it fixes the operator while varying only the residual topology. Plain is strongest on PROTEINS, HorizontalRes is strongest on DD, MatrixResGated is strongest on ENZYMES, and MatrixRes is strongest on MUTAG, AIDS, and Mutagenicity.

Table 4. GCNConv benchmark results with default branch count $B = 3$.

Dataset	Plain	VerticalRes	HorizontalRes	MatrixRes	MatrixResGated
PROTEINS	$\mathbf{0.7044 \pm 0.0387}$	0.6993 ± 0.0340	0.6992 ± 0.0399	0.6973 ± 0.0324	0.6886 ± 0.0397
DD	0.7053 ± 0.0517	0.7143 ± 0.0260	$\mathbf{0.7181 \pm 0.0296}$	0.7127 ± 0.0365	0.7141 ± 0.0359
ENZYMES	0.2683 ± 0.0343	0.2550 ± 0.0455	0.2633 ± 0.0584	0.2750 ± 0.0577	$\mathbf{0.2933 \pm 0.0470}$
MUTAG	0.7556 ± 0.0439	0.7451 ± 0.0695	0.7343 ± 0.0433	$\mathbf{0.7609 \pm 0.0715}$	0.7556 ± 0.0721
AIDS	0.8215 ± 0.0046	0.8210 ± 0.0233	0.8130 ± 0.0297	$\mathbf{0.8365 \pm 0.0412}$	0.8150 ± 0.0231
Mutagenicity	0.7644 ± 0.0188	0.7667 ± 0.0212	0.7738 ± 0.0134	$\mathbf{0.7784 \pm 0.0185}$	0.7701 ± 0.0165

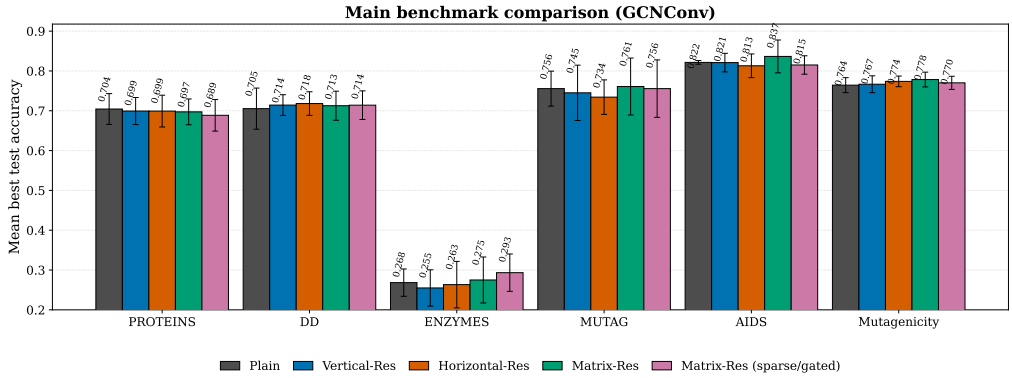

Figure 3. GCNConv benchmark at $B = 3$. Bars show five-fold mean best test accuracy with fold-level standard-deviation error bars.

The GCNConv slice supports the same conclusion as the full benchmark: residual topology matters, but the best topology depends on the dataset. MatrixRes is the strongest default choice on three of six GCNConv datasets, while simpler residual paths are still preferable on PROTEINS and DD. The ENZYMES winner is reported for completeness, but its low absolute accuracy limits the strength of that comparison.

3.3 Branch-count ablation

Branch count has a dataset-dependent effect in the GCNConv branch-count ablation. Table 5 reports the best and worst branch counts for each topology in the branch ablation. On PROTEINS, the useful region is

moderate: HorizontalRes peaks at $B = 4$, MatrixRes peaks at $B = 6$, and MatrixResGated peaks at $B = 3$. On DD, the useful region is smaller: HorizontalRes and MatrixRes peak at $B = 2$, while MatrixResGated is strongest already at $B = 1$.

Table 5. Best and worst branch-count settings in the ablation.

Dataset	Model	Best B	Best accuracy	Worst B	Worst accuracy
PROTEINS	HorizontalRes	4	0.7125 ± 0.0286	1	0.6801 ± 0.0518
PROTEINS	MatrixRes	6	0.7062 ± 0.0210	1	0.6909 ± 0.0586
PROTEINS	MatrixResGated	3	0.7098 ± 0.0219	4	0.6711 ± 0.0471
DD	HorizontalRes	2	0.7275 ± 0.0304	4	0.7071 ± 0.0302
DD	MatrixRes	2	0.7206 ± 0.0411	7	0.7071 ± 0.0390
DD	MatrixResGated	1	0.7283 ± 0.0244	7	0.7053 ± 0.0456

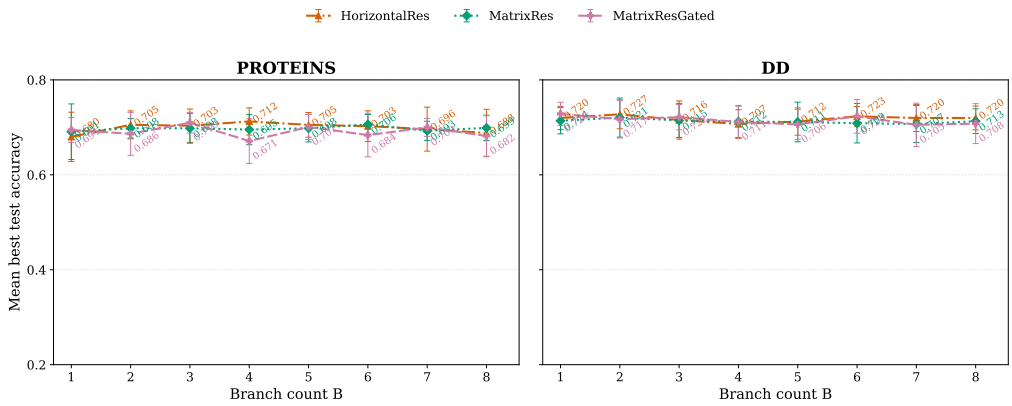

Figure 4. Branch-count ablation for HorizontalRes, MatrixRes, and MatrixResGated on PROTEINS and DD under GCNConv. The best branch budget is dataset- and topology-dependent.

Table 5 and Figure 4 show that branch count should not be read as a simple capacity knob. Increasing B changes the residual graph itself: it adds branches, residual paths, residual traffic, and optimization interactions. PROTEINS tolerates moderate branch expansion, while DD is more sensitive to redundant branch expansion.

3.4 Mechanism analysis: diversity, similarity, gradients, and residual traffic

To interpret the branch-count trends, we analyze branch diversity, branch cosine similarity, branch CKA similarity, representative gradient norms, residual-count totals, active ratios, and gate values. Table 6 reports these mechanism metrics at the best branch count for each ablation target. Diversity measures how far apart branch representations are; branch cosine and branch CKA measure representational redundancy or alignment; gradient norm captures optimization signal; residual count and active ratio describe the amount of residual traffic injected into the model. These measurements are diagnostic correlations rather than causal interventions.

Table 6. Mechanism metrics at the best branch count for each ablation target.

Dataset	Model	B	Acc.	Diversity	Cosine	CKA	Grad.	Residuals	Active	Gate
PROTEINS	HorizontalRes	4	0.7125	1.1450	0.8417	0.9797	0.0157	24.0	0.2519	0.7911
PROTEINS	MatrixRes	6	0.7062	3.7755	0.9173	0.9634	0.0182	88.0	0.4273	0.7757
PROTEINS	MatrixResGated	3	0.7098	1.3555	0.9790	0.9988	0.0316	37.0	0.3121	0.7833
DD	HorizontalRes	2	0.7275	0.1687	0.9969	1.0000	0.0732	6.0	0.3492	0.8375
DD	MatrixRes	2	0.7206	0.1834	0.9991	1.0000	0.0796	14.0	0.5384	0.8222
DD	MatrixResGated	1	0.7283	0.0000	1.0000	1.0000	0.0821	2.0	0.2352	0.8221

The PROTEINS ablation shows that moderate branch separation can be useful. HorizontalRes improves from $B = 1$ to $B = 4$ while branch diversity increases from 0.0000 to 1.1450 and branch cosine

decreases from 1.0000 to 0.8417. However, diversity alone does not guarantee better accuracy: at $B = 8$, diversity rises further to 1.5441, but accuracy falls to 0.6881. MatrixRes follows a similar pattern at a larger scale. It peaks at $B = 6$, where branch diversity reaches 3.7755, but adding still more branches does not improve accuracy.

DD is more conservative. HorizontalRes reaches its best result at $B = 2$, where branch diversity is only 0.1687 and branch cosine remains 0.9969. MatrixRes also peaks at $B = 2$, with similarly high branch cosine (0.9991). Larger branch counts increase diversity on DD, but the performance does not improve. This pattern suggests that larger residual neighborhoods are not useful on DD unless the added branches produce sufficiently useful functional separation.

The sparse/gated model clarifies the role of residual control. MatrixResGated has an active ratio of 0.3121 at the best PROTEINS setting and 0.2352 at the best DD setting. These values are consistent with sparse filtering suppressing residual entries rather than behaving as a dense residual path. At the same time, MatrixResGated does not dominate MatrixRes overall, so sparse control should be treated as a useful traffic-control mechanism rather than as a universal improvement.

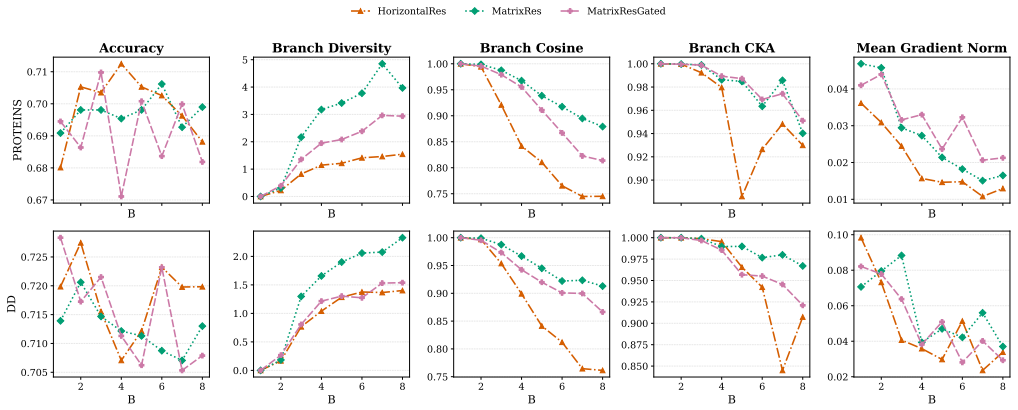

Figure 5. Mechanism summary linking branch-count accuracy to branch diversity, branch cosine similarity, branch CKA, and mean gradient norm.

3.5 MatrixRes and MatrixResGated sensitivity

The fold-0 sensitivity scan identifies candidate operating regions for MatrixRes and MatrixResGated. These scans are used for diagnosis rather than final claims. On PROTEINS, MatrixRes does not improve over its fold-0 baseline in the tested sweep, while MatrixResGated has a fold-0 sparsity candidate: $\lambda = 0.02$ improves fold-0 accuracy from 0.6726 to 0.6951. Stronger sparsification is harmful. On DD, both MatrixRes and MatrixResGated prefer lighter hidden dimensions in the fold-0 scan: MatrixRes reaches 0.7595 with $d = 32$, and MatrixResGated reaches 0.7511 with $d = 32$. MatrixResGated also reaches 0.7511 with $lr = 0.001$ and $gate_init = 0.2$.

Table 7. Best fold-0 sensitivity settings for MatrixRes and MatrixResGated.

Dataset	Model	Baseline	Best setting	Best accuracy
DD	MatrixRes	0.7468	$dim = 32$	0.7595
DD	MatrixResGated	0.7300	$dim = 32$	0.7511
PROTEINS	MatrixRes	0.6637	baseline	0.6637
PROTEINS	MatrixResGated	0.6726	$\lambda = 0.02$	0.6951

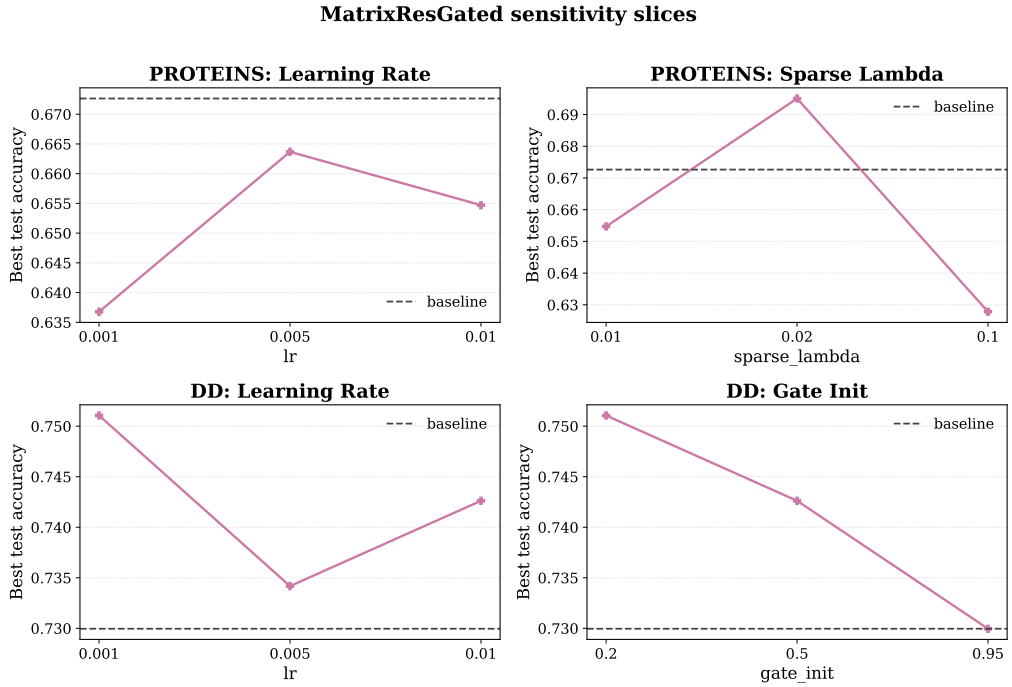

Figure 6. Fold-0 MatrixResGated sensitivity scan at $B = 3$. The scan identifies candidate operating regions but is not treated as final hyperparameter evidence.

3.6 Five-fold check of tuned candidates

To distinguish candidate discovery from evaluation, selected MatrixResGated settings were rerun across all five folds. Table 8 reports these follow-up checks. The strongest practical candidate is `DD_dim_32`, which reduces parameters from 42,371 to 15,043 while remaining competitive. The lower learning rate is close to the default MatrixResGated benchmark, and the lower gate initialization is weaker. On PROTEINS, $\lambda = 0.02$ does not preserve its fold-0 advantage across all folds.

Table 8. Five-fold checks of selected MatrixResGated candidates.

Candidate	Dataset	Accuracy	Mean loss	Parameters
`DD_dim_32`	DD	$\mathbf{0.7215 \pm 0.0330}$	0.5717	15,043
`DD_gate_init_0.2`	DD	0.7131 ± 0.0213	0.5820	42,371
`DD_lr_0.001`	DD	0.7181 ± 0.0336	0.5737	42,371
`PROTEINS_sparse_lambda_0.02`	PROTEINS	0.6909 ± 0.0447	0.6104	38,339

The tuned-candidate check prevents over-claiming from fold-0 sensitivity. It supports a smaller, parameter-efficient MatrixResGated configuration on DD, but it does not support a strong full-fold sparsity claim on PROTEINS.

4 DISCUSSION

This section answers the research questions in light of the six-dataset, four-operator benchmark, the branch-count ablation, the tuned-candidate checks, and the mechanism evidence.

4.1 RQ1: Which residual topologies are preferred?

The main benchmark argues against a universal residual topology. Across 24 dataset–operator combinations, MatrixRes is the most frequent winner and has the best mean rank, but all five model families win at least two combinations. Under GCNConv alone, Plain is strongest on PROTEINS, HorizontalRes is strongest on DD, MatrixResGated is strongest on ENZYMES, and MatrixRes is strongest on MUTAG,

AIDS, and Mutagenicity. These outcomes indicate that the useful residual neighborhood depends on dataset-specific tolerance for branch specialization, residual traffic, and optimization complexity.

This conclusion is consistent with the broader GNN literature: changing the propagation operator, normalization, or pooling design often shifts the preferred architecture (Xu et al., 2019; Hamilton et al., 2017; Veličković et al., 2018; Ying et al., 2018). The present study adds that residual topology itself is another source of this shift. Matrix-style reuse can be read as a local residual stencil over the branch-by-layer grid, analogous in spirit to structured neighborhood mixing (Abu-El-Haija et al., 2019), but applied to hidden-state reuse rather than to graph adjacency. Its advantage is therefore architectural organization, not a new message-passing primitive.

Answer: MatrixRes is the most frequent winner in the current full benchmark, but residual topology is still dataset- and operator-dependent. A graph-classification benchmark should compare vertical, horizontal, matrix-style, and sparse/gated reuse rather than assuming that one residual direction is universally best.

4.2 RQ2: How does branch count change residual behavior?

The branch-count ablation shows that B is not only a capacity parameter. Increasing B adds branches, residual paths, and optimization interactions. In the GCNConv ablation, PROTEINS has a moderate useful region: HorizontalRes peaks at $B = 4$, MatrixRes at $B = 6$, and MatrixResGated at $B = 3$. DD prefers smaller budgets, with peaks at $B = 1$ or $B = 2$. The mechanism summaries are consistent with this split. On PROTEINS, MatrixRes with $B = 6$ produces the largest branch diversity, whereas HorizontalRes keeps lower branch cosine and a smaller residual footprint. On DD, all best branch-count settings have nearly saturated branch cosine and CKA, so extra residual traffic is less likely to create new useful functions. Branch diversity may increase with B, but if branch similarity remains high or gradients weaken, the extra branches do not translate into better classification.

This rise-then-fall behavior is also a practical warning for residual GNN design. Residual connections are often introduced to make deeper or wider models easier to optimize (He et al., 2016; Li et al., 2019, 2021), but a multi-branch residual model changes both capacity and routing density. A larger B increases the number of paths through which hidden states can be reinjected, so the useful regime is governed by the balance between functional diversity and residual interference. The branch-count results suggest that this balance differs sharply between PROTEINS and DD, even though both are commonly treated as protein-structure graph benchmarks.

Answer: In the PROTEINS/DD GCNConv ablation, branch count helps only within a limited operating region. Larger branch budgets are useful when they create functional diversity, and less useful when residual complexity grows faster than branch separation or optimization quality.

4.3 RQ3: When does controlled matrix reuse help?

MatrixResGated tests whether matrix residual traffic should be filtered before injection. The main benchmark shows that sparse/gated reuse is not uniformly better than dense matrix reuse: MatrixRes wins more dataset–operator combinations overall. However, MatrixResGated is still useful because it wins several combinations and provides a controlled way to reduce weak residual coefficients. The ENZYMES GCNConv win is reported but should be interpreted cautiously because all compared models have low absolute accuracy on that dataset. The mechanism table supports the residual-control interpretation: the gated variant generally keeps fewer active residual pathways than dense MatrixRes, while the learned gate remains in a moderate range rather than collapsing to zero. This distinction is important. The value of MatrixResGated is not that sparsity always improves accuracy, but that sparse residual control can be beneficial when matrix-style reuse would otherwise inject too much low-value residual traffic.

This places sparse/gated residual reuse between two familiar design ideas. Like residual gated graph networks, it learns how strongly historical information should be injected (Bresson and Laurent, 2017); like sparsification and regularization methods, it reduces the influence of weak or noisy pathways (Rong et al., 2019; Cangea et al., 2018). The present implementation is deliberately simple: it uses a scalar gate and soft-shrink filtering rather than a large routing network. This keeps the ablation interpretable and prevents the gated variant from becoming a separate high-capacity model.

MatrixResGated defines a tunable family rather than a plug-and-play winner. The fold-0 sensitivity scan suggests candidate operating regions: PROTEINS has a mild-sparsity candidate, while DD has conservative candidates based on smaller hidden dimension and lower initial gate. The five-fold candidate

reruns refine this interpretation. On DD, the smaller hidden dimension remains beneficial and parameter-efficient. On PROTEINS, the mild-sparsity candidate does not preserve its fold-0 advantage across all folds.

Answer: Sparse/gated matrix reuse is a control mechanism for residual traffic. It is complementary to dense MatrixRes rather than a strict replacement, and candidate settings must be validated across folds before they are treated as performance claims. The current evidence supports a smaller controlled matrix model on DD and leaves PROTEINS sparsification as an unstable candidate rather than a confirmed gain.

4.4 Limitations

Several limitations bound the generality of these findings. First, although the main benchmark includes four message-passing operators, it still uses a fixed graph-level pooling and classification head; alternative readout designs may interact with residual topology. Second, the compared operators are canonical message-passing backbones rather than recent graph transformer or local-global hybrid architectures (Dwivedi and Bresson, 2020). This is a deliberate control choice, but it means the paper should not be read as a state-of-the-art model comparison. Third, the sensitivity scan is fold-0 only; although selected candidates were rerun across five folds, that follow-up is limited to MatrixResGated on PROTEINS and DD. Fourth, ENZYMES has low absolute accuracy and high variability, so its result should be interpreted cautiously. Finally, the mechanism analysis is correlational. Branch diversity, cosine similarity, CKA, and gradient norms support a coherent explanation, but they do not prove causality.

Another limitation is that the benchmark intentionally avoids task-specific chemical or biological feature engineering. This makes the residual-topology comparison cleaner, but it also means that the absolute accuracies should not be interpreted as optimized state-of-the-art molecular prediction results. The paper's contribution is methodological: it identifies when different residual pathways are helpful under matched conditions.

4.5 Future directions

Future work should extend the comparison to larger graph-classification datasets, richer readout mechanisms, recent local-global backbones, and adaptive residual-neighborhood selection. A natural next step is to replace manually chosen vertical, horizontal, or matrix stencils with a learned local residual policy. The tuned-candidate validation should also be broadened beyond the current MatrixResGated checks to determine whether dataset-specific residual control can be selected reliably.

5 CONCLUSIONS

This paper studies residual connectivity for graph classification as a structured branch-by-layer design problem. Across six datasets and four message-passing operators, residual topology matters, but its value depends on the dataset, operator, and branch budget. MatrixRes is the most frequent winner and has the best mean rank in the full benchmark, but the per-dataset GCNConv winners remain mixed: Plain is strongest on PROTEINS, HorizontalRes on DD, MatrixResGated on ENZYMES, and MatrixRes on MUTAG, AIDS, and Mutagenicity. The ENZYMES comparison has low absolute accuracy and should be interpreted cautiously. The branch-count ablations further show that adding branches is useful only within a limited operating region.

The mechanism analysis provides a plausible explanation for why branch expansion is not automatically beneficial. Additional branches can increase representational diversity, but they also increase residual traffic and may preserve redundant branch similarity or weaken optimization. Matrix-style reuse is best viewed as a controlled residual family whose effectiveness depends on the branch budget and residual filtering strategy.

The practical implication is that residual topology, branch count, operator choice, and residual control should be evaluated jointly. Before treating a sensitivity result as a performance claim, candidate settings should be rerun across folds. In the current experiments, this validation supports a smaller and more parameter-efficient MatrixResGated model on DD, while the PROTEINS sparsity candidate remains less stable.

6 ACKNOWLEDGMENTS

The authors thank the open-source graph learning community and the maintainers of the benchmark resources used in this study. They also thank the reviewers and editors for their time and feedback.

A APPENDIX

A.1 Result sources

The manuscript uses the following summary files as the source of truth. The main benchmark summary contains 120 model-level rows: six datasets, four operators, five model families, and five-fold aggregation for each combination.

- `records/LATEST/summaries/benchmark_summary.csv`
- `records/LATEST/summaries/branch_ablation_summary.csv`
- `records/LATEST/summaries/parameter_sensitivity_summary.csv`
- `records/LATEST/summaries/tuned_candidate_summary.csv`
- `records/LATEST/summaries/mechanism_compact_summary.csv`

A.2 Supplementary diagnostics

Additional fold-level rows, CKA summaries, residual traffic summaries, and full sensitivity tables are retained in the repository records directory for supplementary conversion.

A.3 Evidence map

Table 9 summarizes where each main empirical claim is supported in the manuscript. This compact table is included to keep the main text readable while still making the result-to-claim link explicit.

Table 9. Compact map from empirical claims to manuscript evidence.

Claim	Primary evidence
MatrixRes is the most frequent winner in the full benchmark but not a universal winner.	Table 2, Table 3, and Figure 2.
GCNConv results remain dataset-dependent under a fixed operator.	Table 4 and Figure 3.
Branch count produces a rise-then-fall pattern rather than monotonic improvement.	Table 5 and Figure 4.
Useful branch expansion is associated with separation without excessive residual traffic.	Table 6 and Figure 5.
Fold-0 sensitivity should not be treated as final evidence.	Table 7, Figure 6, and Table 8.

REFERENCES

Sami Abu-El-Haija, Bryan Perozzi, Aalok Kapoor, Navid Alipourfard, Kristina Lerman, Greg Steeg, and Aram Galstyan. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *International Conference on Machine Learning*, pages 33–42, 2019.

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.

Filippo Maria Bianchi, Franco Grattarola, and Cesare Alippi. Graph neural networks with convolutional arma filters. In *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020.

Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.

Cătălina Cangea, Petar Veličković, Nikola Jovanović, Thomas Kipf, and Pietro Liò. Towards sparse hierarchical graph classifiers. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pages 165–176, 2018.

David K Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alan Aspuru-Guzik, and Ryan P Adams. Convolutional networks on graphs for learning molecular fingerprints. In *Advances in neural information processing systems*, pages 2224–2232, 2015.

Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs. *arXiv preprint arXiv:2012.09699*, 2020.

Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.

Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.

Marco Gori, Gabriele Monfardini, and Franco Scarselli. A new model for learning in graph domains. *Proceedings of IJCNN*, 2005.

William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Nicolas Keriven, Gabriel Peyré, and Samuel Vaiter. A unified benchmark for the diagnosis and treatment of graph neural networks. *arXiv preprint arXiv:2012.01450*, 2020.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017. URL `https://arxiv.org/abs/1609.02907`.

Junhyun Lee, Inyeop Lee, and Jaewoo Kang. Self-attention graph pooling. *arXiv preprint arXiv:1904.08082*, 2019.

Guohao Li, Matthias Müller, Ali Thabet, and Bernard Ghanem. DeepGCNs: Can GCNs go as deep as CNNs? In *IEEE/CVF International Conference on Computer Vision*, 2019. URL `https://arxiv.org/abs/1904.03751`.

Guohao Li, Elias Müller, Abbas Thabet, and Bernard Ghanem. Training graph neural networks with 1000 layers. In *International Conference on Machine Learning*, pages 6403–6413, 2021.

Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. URL `https://ojs.aaai.org/index.php/AAAI/article/view/11604`.

Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. TUDataset: A collection of benchmark datasets for learning with graphs. In *ICML Workshop on Graph Representation Learning and Beyond*, 2020. URL `https://chrsmrrs.github.io/datasets/`.

Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint arXiv:2006.13604*, 2020.

Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.

Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe Vendrig, and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guoding Long, Cheng Zhang, and Sheng Yu Zhang. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*, pages 5449–5458, 2018.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL `https://openreview.net/forum?id=ryGs6iA5Km`.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in neural information processing systems*, pages 4800–4810, 2018.